



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Parallel Programming

Prof. Marco Bertini



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Shared memory: OpenMP optimizations



Memory and thread allocation

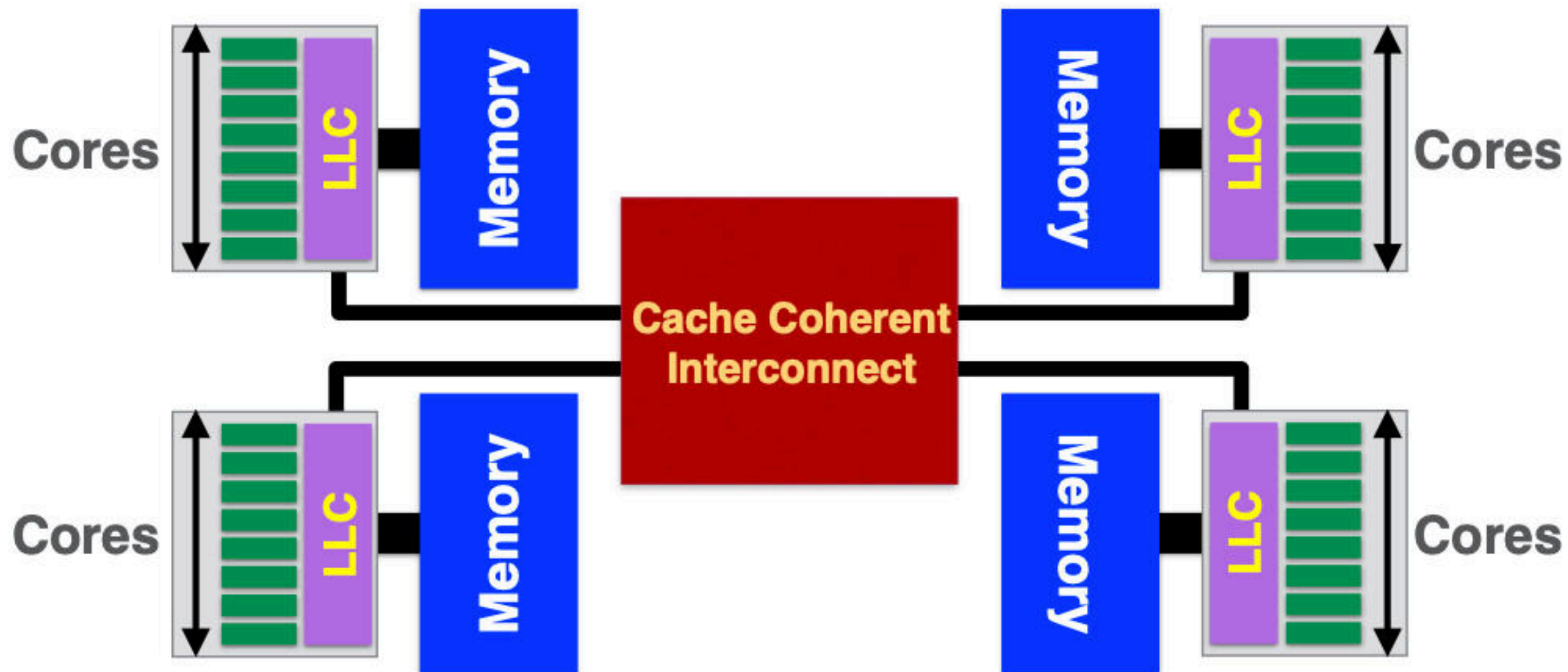


Allocating memory

- The OS kernel can use several techniques to manage memory for OpenMP and threading.
The most common technique is the **first touch** concept, where memory is allocated nearest to the thread where it is first touched.
- The first touch of an array causes the memory to be allocated. The memory is allocated near the thread location where the touch occurs. Prior to the first touch, the memory only exists as an entry in a virtual memory table. The physical memory that corresponds to the virtual memory is created when it is first accessed.
- Policy used on Linux

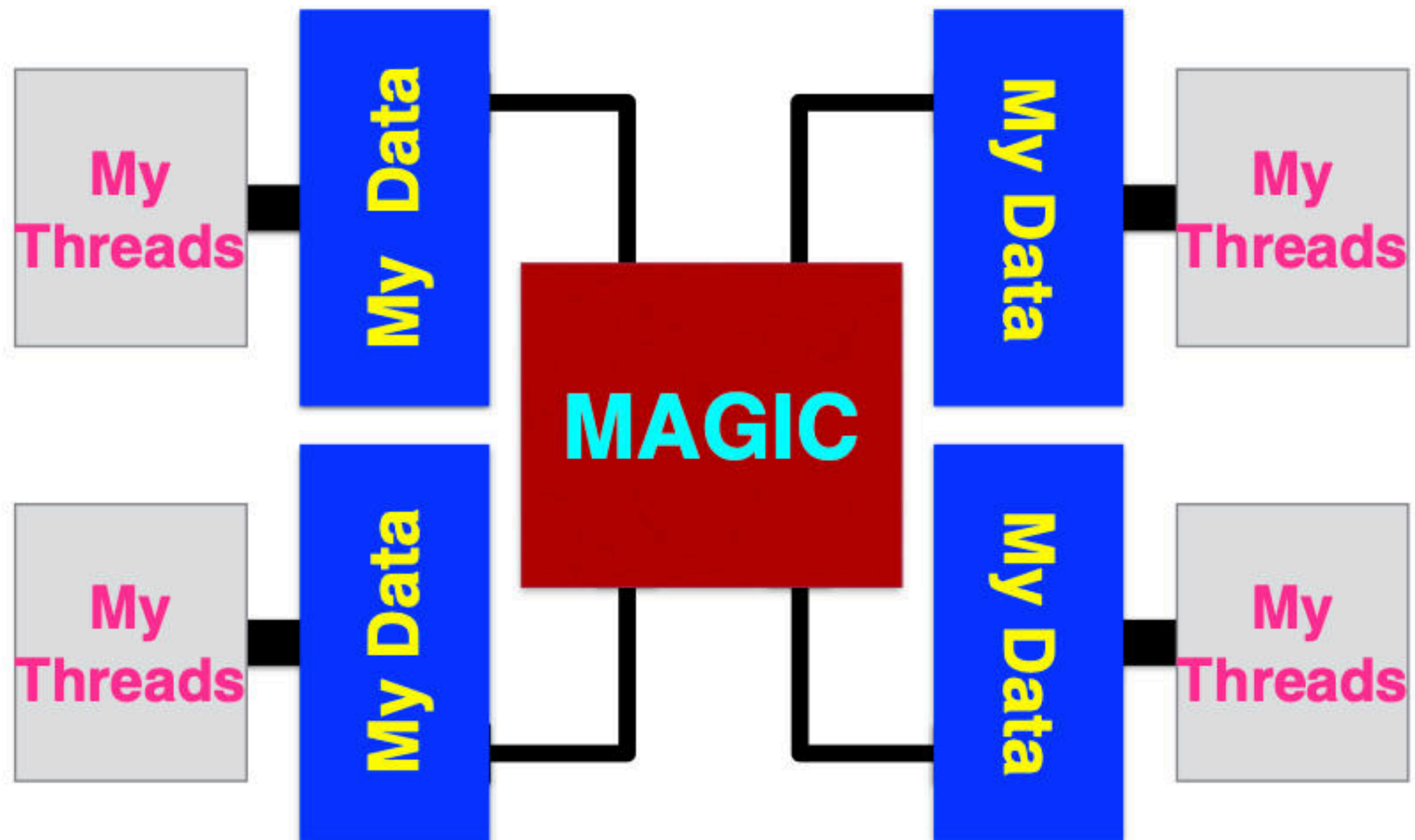
NUMA systems

- Modern parallel computers are based on NUMA

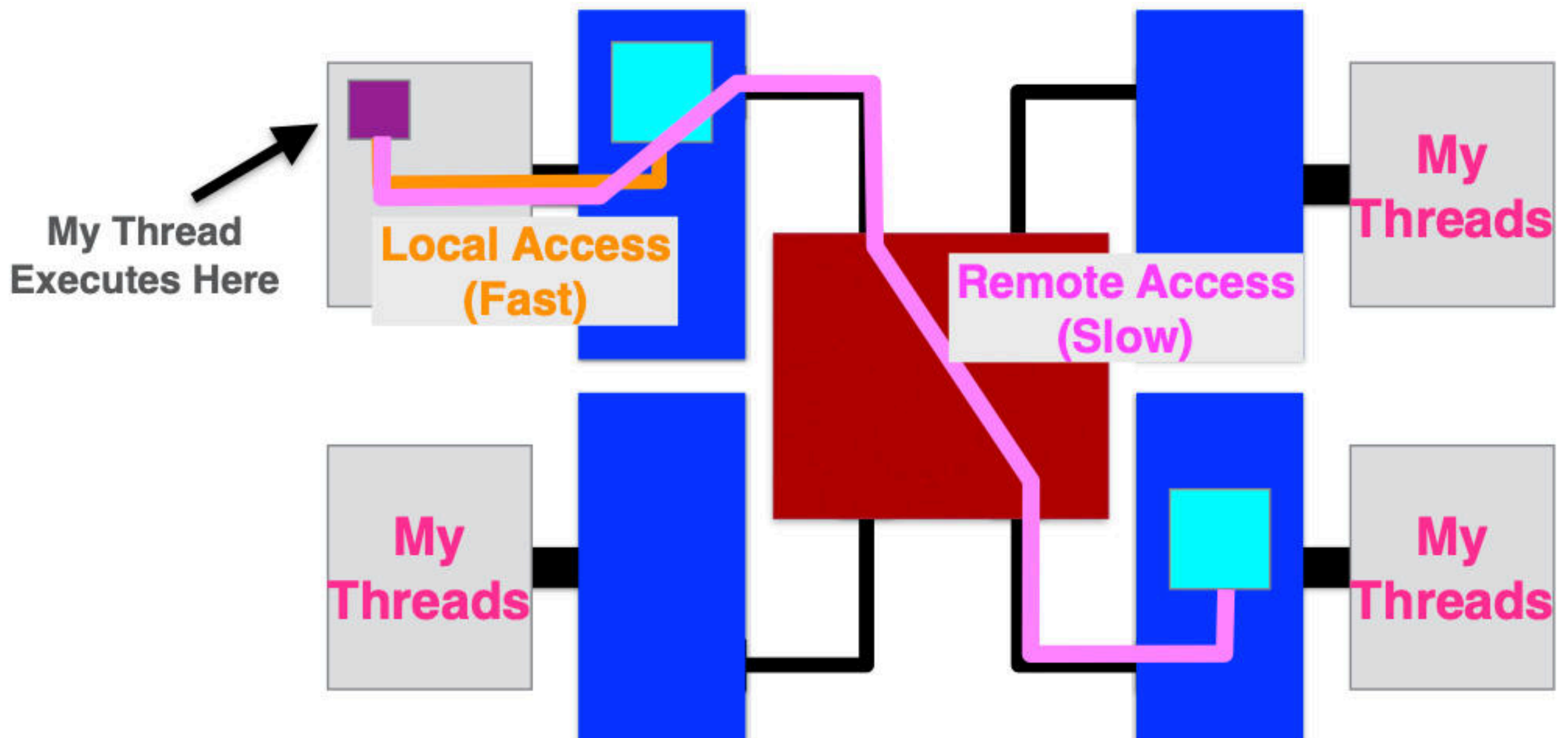


NUMA systems

- Modern parallel computers are based on NUMA



NUMA systems



numactl

- Get NUMA details with numactl command, e.g. with numactl -H:

available: 2 nodes (0-1) Puoi vedere che succede nel caso di macchine a più cores/nodi
node 0 cpus: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77

node 0 size: 385386 MB
node 0 free: 384584 MB

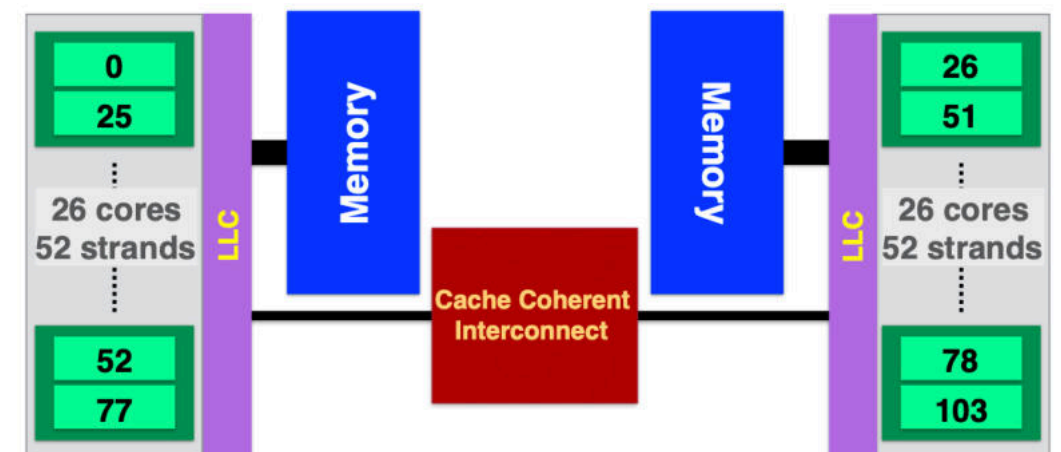
node 1 cpus: 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48
49 50 51 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
101 102 103

node 1 size: 387061 MB
node 1 free: 386796 MB

node distances:

node	0	1
0:	10	21
1:	21	10

- The distances table show the relative latencies



LLC = last level cache

Memory allocation and first touch

- If a thread allocates all the memory then all the data will be stored on a single node. Other CPUs will have slow access to it.
- Solution: parallelize data initialization, e.g.:

```
#pragma omp parallel for \\  
    schedule(static)  
for (int i=0; i<n; i++)  
    a[i] = 0;
```

- each thread has a slice of “a” in its local memory

Example w/o first touch

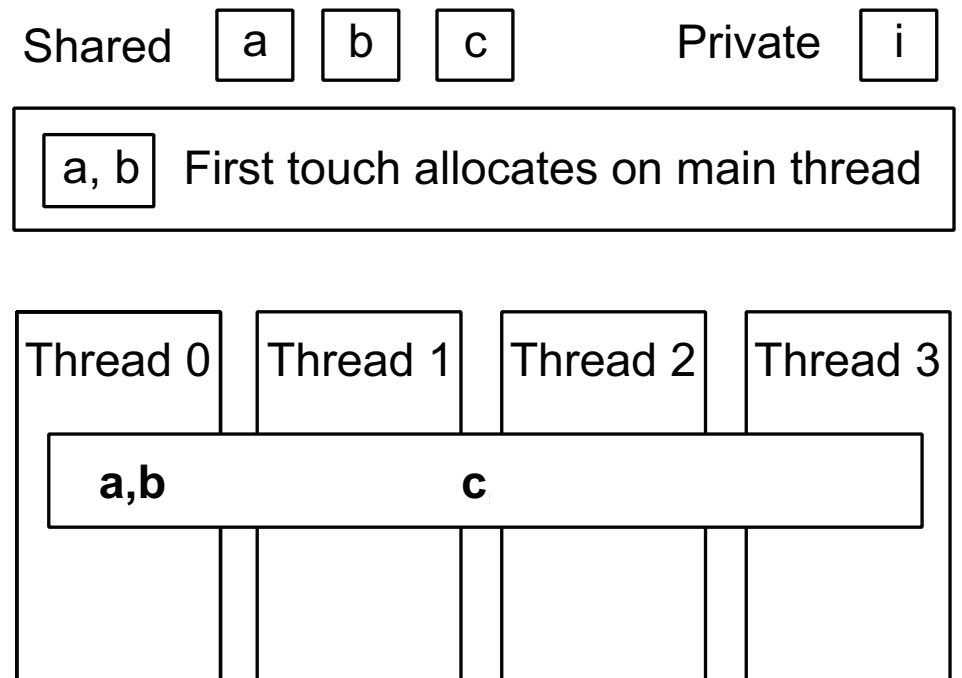
```
#define ARRAY_SIZE 80000000
static double a[ARRAY_SIZE], b[ARRAY_SIZE], c[ARRAY_SIZE];
```

Notare lo static

```
// ...
for (int i=0; i<ARRAY_SIZE; i++) {
    a[i] = 1.0;
    b[i] = 2.0;
}
```

// ...

```
#pragma omp parallel for
for (int i=0; i < n; i++) {
    c[i] = a[i] + b[i];
}
```



Thread 0 ha molto più facile accesso ad `a` e `b` rispetto agli altri thread a quali viene assegnato `c`

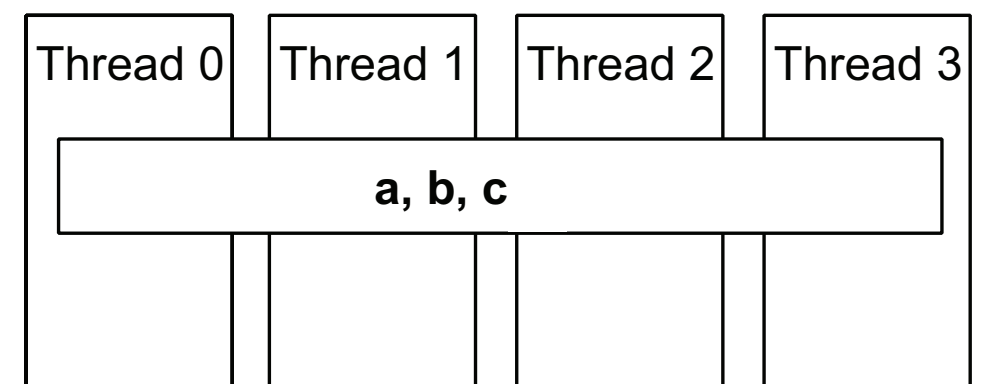
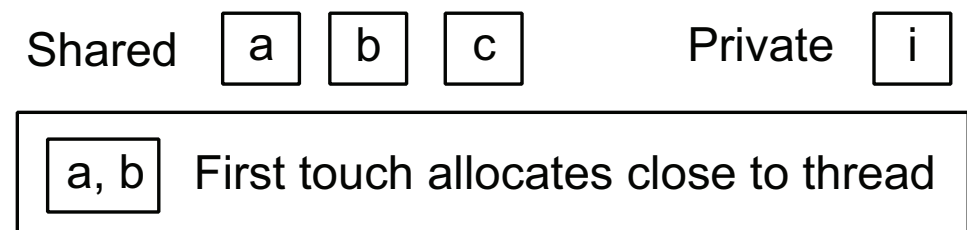
Example with first touch

```
#define ARRAY_SIZE 80000000
static double a[ARRAY_SIZE], b[ARRAY_SIZE], c[ARRAY_SIZE];
```

```
// ...
#pragma omp parallel for
for (int i=0; i<ARRAY_SIZE; i++) {
    a[i] = 1.0;
    b[i] = 2.0;
}
```

```
// ...

#pragma omp parallel for
for (int i=0; i < n; i++) {
    c[i] = a[i] + b[i];
}
```



Thread affinity

- Tying a thread to the location of the memory it uses is important to achieve good memory latency and bandwidth.
- Two OpenMP environment variables help to control how threads are allocated and moved
- OMP_PLACES - Defines where threads may run
- OMP_PROC_BIND - Defines how threads map onto the OpenMP places

Thread affinity

Rilevante per aumentare lo speedup... se thread rientra su zona di memoria simile (con simili dati) v'è più veloce!

- `export OMP_PLACES=cores`
Threads are scheduled on the system cores

`omp_get_num_places()` returns the number of available places
- `export OMP_PROC_BIND=spread`
Threads should be placed on cores as far away from each other as possible

`omp_get_proc_bind()` reports the policy used

OMP_PLACES values

- sockets [$\langle n \rangle$] - Threads are scheduled on sockets
- cores [$\langle n \rangle$] - Threads are scheduled on cores
- threads [$\langle n \rangle$] - Threads are scheduled on strands (i.e. hardware threads, a CPU core with hyperthreading has two “strands”)
- “user defined set” - Use strand IDs to schedule threads, e.g. “ $\{0\}, \{8\}, \{16\}, \{24\}$ ”

OMP_PLACES values

- sockets [$\langle n \rangle$] - Threads are scheduled on sockets
- cores [$\langle n \rangle$] - Threads are scheduled on cores
- threads [$\langle n \rangle$] - Threads are scheduled on strands (i.e. hardware threads, a CPU core with hyperthreading has two “strands”)

In this context strand is used to differentiate between software threads of OpenMP from hardware threads of the CPU

- “user defined set” - Use strand IDs to schedule threads, e.g. “ $\{0\}$, $\{8\}$, $\{16\}$, $\{24\}$ ”

OMP_PROC_BIND values

- Since OpenMP 4.0:
 - `master` - Schedule threads in the same place where the master thread is executing. Since OpenMP 5.1 it is renamed `primary`
 - `close` - Keep threads “close” in terms of the places
 - `spread` - Spread threads as far as possible in terms of the places
- Spread può essere un'idea nel caso di aggiornamenti dei valori...
per evitare magari conflitti

OMP_PROC_BIND values

- Alternatively, and before OpenMP 4.0
- `true` - the kernel does not to move the thread once it gets scheduled.
- `false` - the kernel scheduler is free to move threads around



Rules of thumb

- Hyperthreading does not help with simple memory-bound kernels (i.e. functions), but it also doesn't hurt.
- For memory-bandwidth-limited kernels on multiple sockets (NUMA domains), getting the sockets busy helps (e.g. using `OMP_PROC_BIND=spread`).
- Consider the first-touch policy to allocate memory, it may greatly improve performance (e.g. 10x)



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Optimizing high-level OpenMP

Optimization process

- Base implementation — Implement simple loop-level OpenMP
- Step 1: Reduce thread start up — Merge the parallel regions and join all the loop-level parallel constructs into larger parallel regions
- Step 2: Synchronization — Add nowait clauses to for loops, where synchronization is not needed, and calculate and manually partition the loops across the threads, which allows for removal of barriers and required synchronization.
- Step 3: Optimize — Make arrays and variables private to each thread when possible.
Evito sicuramente race conditions e quindi eventuali controlli
- Step 4: Code correctness — Check thoroughly for race conditions (after every step). Use tools like Intel Inspector.

Step 1

- Go from

```
#pragma omp parallel for
```

to

```
#pragma omp parallel
```

```
#pragma omp for
```

- OpenMP overhead may account for 10-15% of execution time

Step 2

- Whenever possible add `nowait` to reduce synchronization costs due to the implicit barriers (e.g. in for loops)
- A further optimization is to avoid the work sharing of for loops by explicitly computing the work assignment based on thread ids, e.g. using something like:

```
tbegin = N * threadID / nthreads  
tend   = N * (threadID+1) / nthreads
```

- The impact of the manual partitioning of the arrays is that it reduces cache thrashing and race conditions by not allowing threads to share the same space in memory

Step 3

- Privatization of variables avoids synchronization
- Helps compiler optimization since it does not have to consider dependencies

Più aggiungo private e meglio è...
ridurre il più possibile variabili shared

Scoping rules

Specification		Shared	Private	Reduction
Parallel construct		Variables declared outside parallel construct or in a shared clause	Automatic variables within parallel construct or in a private or firstprivate clause	Reduction clause
Parallel region	Fortran routine	save attribute, initialized variables, common block, module variables	All local variables or declared threadprivate. Also dynamically allocated	
	C routine	File scope variables, extern, or static		
	Arguments	Inherited from calling environment		
Always		All loop indices will be private		

Optimization example

- Let's consider some code to implement a stencil operation.

```
int imax=2002, jmax = 2002;

double** xtmp;
double** x = malloc2D(jmax, imax);
double** xnew = malloc2D(jmax, imax);
int *flush = (int *)malloc(jmax*imax*sizeof(int)*4);

cpu_timer_start(&tstart_total);
cpu_timer_start(&tstart_init);
#pragma omp parallel for
for (int j = 0; j < jmax; j++){
    for (int i = 0; i < imax; i++){
        xnew[j][i] = 0.0;
        x[j][i] = 5.0;
    }
}

#pragma omp parallel for
for (int j = jmax/2 - 5; j < jmax/2 + 5; j++){
    for (int i = imax/2 - 5; i < imax/2 - 1; i++){
        x[j][i] = 400.0;
    }
}
```

```
double **malloc2D(int jmax, int imax)
{
    // first allocate a block of memory for the row pointers and the 2D array
    double **x = (double **)malloc(jmax*sizeof(double *) + jmax*imax*sizeof(double));

    // Now assign the start of the block of memory for the 2D array after the row pointers
    x[0] = (double *)x + jmax;

    // Last, assign the memory location to point to for each row pointer
    for (int j = 1; j < jmax; j++) {
        x[j] = x[j-1] + imax;
    }

    return(x);
}
```

Optimization example

- Let's consider some code to implement a stencil operation.

```
int imax=2002, jmax = 2002;

double** xtmp;
double** x = malloc2D(jmax, imax);
double** xnew = malloc2D(jmax, imax);
int *flush = (int *)malloc(jmax*imax*sizeof(int)*4);

cpu_timer_start(&tstart_total);
cpu_timer_start(&tstart_init);
#pragma omp parallel for
for (int j = 0; j < jmax; j++){
    for (int i = 0; i < imax; i++){
        xnew[j][i] = 0.0;
        x[j][i] = 5.0;
    }
}

#pragma omp parallel for
for (int j = jmax/2 - 5; j < jmax/2 + 5; j++){
    for (int i = imax/2 - 5; i < imax/2 - 1; i++){
        x[j][i] = 400.0;
    }
}
```

```
double **malloc2D(int jmax, int imax)
{
    // first allocate a block of memory for the row pointers and the 2D array
    double **x = (double **)malloc(jmax*sizeof(double *) + jmax*imax*sizeof(double));

    // Now assign the start of the block of memory for the 2D array after the row pointers
    x[0] = (double *)x + jmax;

    // Last, assign the memory location to point to for each row pointer
    for (int j = 1; j < jmax; j++) {
        x[j] = x[j-1] + imax;
    }

    return(x);
}
```

First-touch initialization




```
#pragma omp parallel for
for (int j = 0; j < jmax; j++){
    for (int i = 0; i < imax; i++){
        xnew[j][i] = 0.0;
        x[j][i] = 5.0;
    }
}

#pragma omp parallel for
for (int j = jmax/2 - 5; j < jmax/2 + 5; j++){
    for (int i = imax/2 - 5; i < imax/2 - 1; i++){
        x[j][i] = 400.0;
    }
}

init_time += cpu_timer_stop(tstart_init);

for (int iter = 0; iter < 10000; iter++){
    cpu_timer_start(&tstart_flush);
    #pragma omp parallel for
    for (int l = 1; l < jmax*imax*4; l++){
        flush[l] = 1.0;
    }
    flush_time += cpu_timer_stop(tstart_flush);
    cpu_timer_start(&tstart_stencil);
    #pragma omp parallel for
    for (int j = 1; j < jmax-1; j++){
        for (int i = 1; i < imax-1; i++){
            xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j-1][i] + x[j+1][i] )/5.0;
        }
    }
    stencil_time += cpu_timer_stop(tstart_stencil);

    SWAP_PTR(xnew, x, xtmp);
    if (iter%1000 == 0) printf("Iter %d\n",iter);
}
```

```
#pragma omp parallel for
for (int j = 0; j < jmax; j++){
    for (int i = 0; i < imax; i++){
        xnew[j][i] = 0.0;
        x[j][i] = 5.0;
    }
}

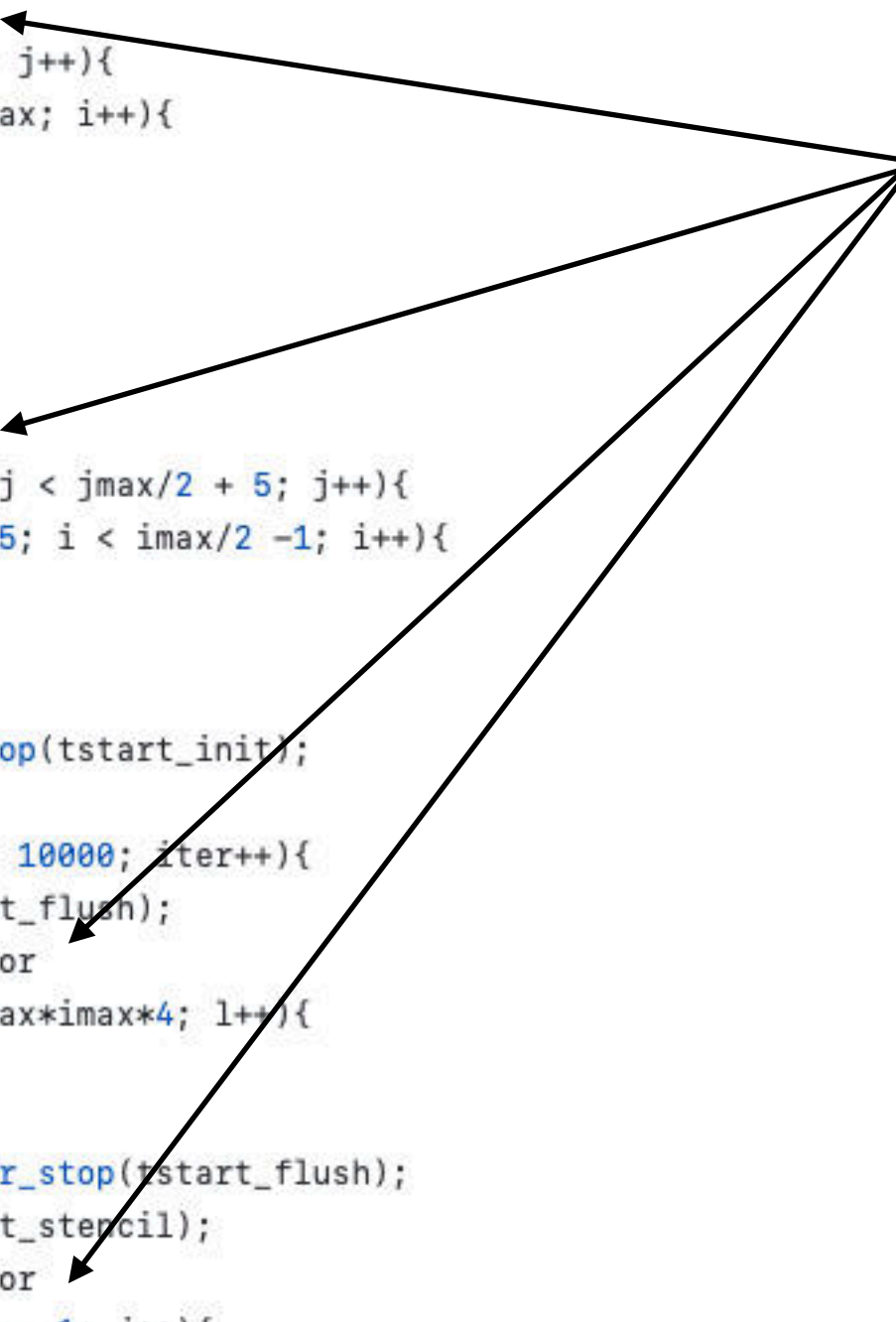
#pragma omp parallel for
for (int j = jmax/2 - 5; j < jmax/2 + 5; j++){
    for (int i = imax/2 - 5; i < imax/2 - 1; i++){
        x[j][i] = 400.0;
    }
}

init_time += cpu_timer_stop(tstart_init);

for (int iter = 0; iter < 10000; iter++){
    cpu_timer_start(&tstart_flush);
    #pragma omp parallel for
    for (int l = 1; l < jmax*imax*4; l++){
        flush[l] = 1.0;
    }
    flush_time += cpu_timer_stop(tstart_flush);
    cpu_timer_start(&tstart_stencil);
    #pragma omp parallel for
    for (int j = 1; j < jmax-1; j++){
        for (int i = 1; i < imax-1; i++){
            xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j-1][i] + x[j+1][i] )/5.0;
        }
    }
    stencil_time += cpu_timer_stop(tstart_stencil);

    SWAP_PTR(xnew, x, xtmp);
    if (iter%1000 == 0) printf("Iter %d\n",iter);
}
```

Spawn threads



```

#pragma omp parallel for
for (int j = 0; j < jmax; j++){
    for (int i = 0; i < imax; i++){
        xnew[j][i] = 0.0;
        x[j][i] = 5.0;
    }
}

#pragma omp parallel for
for (int j = jmax/2 - 5; j < jmax/2 + 5; j++){
    for (int i = imax/2 - 5; i < imax/2 - 1; i++){
        x[j][i] = 400.0;
    }
}

init_time += cpu_timer_stop(tstart_init);

for (int iter = 0; iter < 10000; iter++){
    cpu_timer_start(&tstart_flush);
    #pragma omp parallel for
    for (int l = 1; l < jmax*imax*4; l++){
        flush[l] = 1.0;
    }
    flush_time += cpu_timer_stop(tstart_flush);
    cpu_timer_start(&tstart_stencil);
    #pragma omp parallel for
    for (int j = 1; j < imax-1; j++){
        for (int i = 1; i < imax-1; i++){
            xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j-1][i] + x[j+1][i] )/5.0;
        }
    }
    stencil_time += cpu_timer_stop(tstart_stencil);

    SWAP_PTR(xnew, x, xtmp);
    if (iter%1000 == 0) printf("Iter %d\n",iter);
}

```

Spawn threads

Implicit barrier.
Join threads.

Un po' troppi fork joins...


```
#pragma omp parallel for
for (int j = 0; j < jmax; j++){
    for (int i = 0; i < imax; i++){
        xnew[j][i] = 0.0;
        x[j][i] = 5.0;
    }
}

#pragma omp parallel for
for (int j = jmax/2 - 5; j < jmax/2 + 5; j++){
    for (int i = imax/2 - 5; i < imax/2 - 1; i++){
        x[j][i] = 400.0;
    }
}

init_time += cpu_timer_stop(tstart_init);

for (int iter = 0; iter < 10000; iter++){
    cpu_timer_start(&tstart_flush);
    #pragma omp parallel for
    for (int l = 1; l < jmax*imax*4; l++){
        flush[l] = 1.0;
    }
    flush_time += cpu_timer_stop(tstart_flush);
    cpu_timer_start(&tstart_stencil);
    #pragma omp parallel for
    for (int j = 1; j < imax-1; j++){
        for (int i = 1; i < imax-1; i++){
            xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j-1][i] + x[j+1][i] )/5.0;
        }
    }
    stencil_time += cpu_timer_stop(tstart_stencil);

    SWAP_PTR(xnew, x, xtmp);
    if (iter%1000 == 0) printf("Iter %d\n",iter);
}
```

Spawn threads

Implicit barrier.
Join threads.

What is it for? It's to evaluate the performance of the program in a cold cache scenario. It mimics the performance in the case of cache misses for **x** and **xnew**. It is useful if you run the code within a loop to get better measurements

```
#pragma omp parallel
{
    int thread_id = omp_get_thread_num();
    for (int iter = 0; iter < 10000; iter++){
        if (thread_id == 0) cpu_timer_start(&tstart_flush);
        #pragma omp for nowait
        for (int l = 1; l < jmax*imax*4; l++){
            flush[l] = 1.0;
        }
        if (thread_id == 0){
            flush_time += cpu_timer_stop(tstart_flush);
            cpu_timer_start(&tstart_stencil);
        }
        #pragma omp for
        for (int j = 1; j < jmax-1; j++){
            for (int i = 1; i < imax-1; i++){
                xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j-1][i] + x[j+1][i] )/5.0;
            }
        }
        if (thread_id == 0){
            stencil_time += cpu_timer_stop(tstart_stencil);

            SWAP_PTR(xnew, x, xtmp);
            if (iter%1000 == 0) printf("Iter %d\n",iter);
        }
    }
} // end omp parallel
```

Spawn threads

```
#pragma omp parallel
{
    int thread_id = omp_get_thread_num();
    for (int iter = 0; iter < 10000; iter++){
        if (thread_id == 0) cpu_timer_start(&tstart_flush);
        #pragma omp for nowait
        for (int l = 1; l < jmax*imax*4; l++){
            flush[l] = 1.0;
        }
        if (thread_id == 0){
            flush_time += cpu_timer_stop(tstart_flush);
            cpu_timer_start(&tstart_stencil);
        }
        #pragma omp for
        for (int j = 1; j < jmax-1; j++){
            for (int i = 1; i < imax-1; i++){
                xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j-1][i] + x[j+1][i] )/5.0;
            }
        }
        if (thread_id == 0){
            stencil_time += cpu_timer_stop(tstart_stencil);

            SWAP_PTR(xnew, x, xtmp);
            if (iter%1000 == 0) printf("Iter %d\n",iter);
        }
    }
} // end omp parallel
```



```
#pragma omp parallel
```

Spawn threads

```
{  
    int thread_id = omp_get_thread_num();  
    for (int iter = 0; iter < 10000; iter++){  
        if (thread_id == 0) cpu_timer_start(&tstart_flush);  
        #pragma omp for nowait  
        for (int l = 1; l < jmax*imax*4; l++){  
            flush[l] = 1.0;  
        }  
        if (thread_id == 0){  
            flush_time += cpu_timer_stop(tstart_flush);  
            cpu_timer_start(&tstart_stencil);  
        }  
        #pragma omp for  
        for (int j = 1; j < jmax-1; j++){  
            for (int i = 1; i < imax-1; i++){  
                xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j-1][i] + x[j+1][i] )/5.0;  
            }  
        }  
        if (thread_id == 0){  
            stencil_time += cpu_timer_stop(tstart_stencil);  
  
            SWAP_PTR(xnew, x, xtmp);  
            if (iter%1000 == 0) printf("Iter %d\n",iter);  
        }  
    }  
} // end omp parallel
```

Un solo fork - join!

Implicit barrier.
Join threads

`#pragma omp parallel` ← Spawn threads

{

`int thread_id = omp_get_thread_num();`

`for (int iter = 0; iter < 10000; iter++){`

`if (thread_id == 0) cpu_timer_start(&tstart_flush);`

`#pragma omp for nowait` ← No synchronization (not required)

`for (int l = 1; l < jmax*imax*4; l++){`

`flush[l] = 1.0;`

`}`

`if (thread_id == 0){`

`flush_time += cpu_timer_stop(tstart_flush);`

`cpu_timer_start(&tstart_stencil);`

`}`

`#pragma omp for`

`for (int j = 1; j < jmax-1; j++){`

`for (int i = 1; i < imax-1; i++){`

`xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j-1][i] + x[j+1][i] )/5.0;`

`}`

`}`

`if (thread_id == 0){`

`stencil_time += cpu_timer_stop(tstart_stencil);`

`SWAP_PTR(xnew, x, xtmp);`

`if (iter%1000 == 0) printf("Iter %d\n",iter);`

`}`

`}`

`} // end omp parallel` ← Implicit barrier.
Join threads

```
#pragma omp parallel
{
    int thread_id = omp_get_thread_num();
    int nthreads = omp_get_num_threads();

    int jltb = 1 + (jmax-2) * ( thread_id      ) / nthreads;
    int jutb = 1 + (jmax-2) * ( thread_id + 1 ) / nthreads;

    int ifltb = (jmax*imax*4) * ( thread_id      ) / nthreads;
    int ifutb = (jmax*imax*4) * ( thread_id + 1 ) / nthreads;

    int jltb0 = jltb;
    if (thread_id == 0) jltb0--;
    int jutb0 = jutb;
    if (thread_id == nthreads-1) jutb0++;

    int kmin = MAX(jmax/2-5,jltb);
    int kmax = MIN(jmax/2+5,jutb);

    if (thread_id == 0) cpu_timer_start(&tstart_init);
    for (int j = jltb0; j < jutb0; j++){
        for (int i = 0; i < imax; i++){
            xnew[j][i] = 0.0;
            x[j][i] = 5.0;
        }
    }

    for (int j = kmin; j < kmax; j++){
        for (int i = imax/2 - 5; i < imax/2 -1; i++){
            x[j][i] = 400.0;
        }
    }
    #pragma omp barrier
    if (thread_id == 0) init_time += cpu_timer_stop(tstart_init);
}
```

Approccio alternativo...
scrivo a mano i thread da associare


```
#pragma omp parallel
{
    int thread_id = omp_get_thread_num();
    int nthreads = omp_get_num_threads();

    int jltb = 1 + (jmax-2) * ( thread_id ) / nthreads;
    int jutb = 1 + (jmax-2) * ( thread_id + 1 ) / nthreads;

    int ifltb = (jmax*imax*4) * ( thread_id ) / nthreads;
    int ifutb = (jmax*imax*4) * ( thread_id + 1 ) / nthreads;

    int jltb0 = jltb;
    if (thread_id == 0) jltb0--;
    int jutb0 = jutb;
    if (thread_id == nthreads-1) jutb0++;

    int kmin = MAX(jmax/2-5,jltb);
    int kmax = MIN(jmax/2+5,jutb);

    if (thread_id == 0) cpu_timer_start(&tstart_init);
    for (int j = jltb0; j < jutb0; j++){
        for (int i = 0; i < imax; i++){
            xnew[j][i] = 0.0;
            x[j][i] = 5.0;
        }
    }

    for (int j = kmin; j < kmax; j++){
        for (int i = imax/2 - 5; i < imax/2 -1; i++){
            x[j][i] = 400.0;
        }
    }
    #pragma omp barrier
    if (thread_id == 0) init_time += cpu_timer_stop(tstart_init);
}
```

Computation of the loop bounds.
The input matrix is divided in horizontal stripes, each one is assigned to a thread (jltb/jutb).

Magic numbers:
2=half of the (stencil size-1)
5=stencil size.

Passo a variabili private...
no race conditions e quindi non hanno bisogno di sync
Variabili scritte nei registri! ... più ottimizzate


```
#pragma omp parallel
```

```
{
```

```
int thread_id = omp_get_thread_num();
int nthreads = omp_get_num_threads();

int jltb = 1 + (jmax-2) * ( thread_id      ) / nthreads;
int jutb = 1 + (jmax-2) * ( thread_id + 1 ) / nthreads;

int ifltb = (jmax*imax*4) * ( thread_id      ) / nthreads;
int ifutb = (jmax*imax*4) * ( thread_id + 1 ) / nthreads;

int jltb0 = jltb;
if (thread_id == 0) jltb0--;
int jutb0 = jutb;
if (thread_id == nthreads-1) jutb0++;

int kmin = MAX(jmax/2-5,jltb);
int kmax = MIN(jmax/2+5,jutb);
```

```
if (thread_id == 0) cpu_timer_start(&tstart_init);
```

```
for (int j = jltb0; j < jutb0; j++){
    for (int i = 0; i < imax; i++){
        xnew[j][i] = 0.0;
        x[j][i] = 5.0;
    }
}

for (int j = kmin; j < kmax; j++){
    for (int i = imax/2 - 5; i < imax/2 -1; i++){
        x[j][i] = 400.0;
    }
}
```

```
#pragma omp barrier
```

```
if (thread_id == 0) init_time += cpu_timer_stop(tstart_init);
```

Computation of the loop bounds.
The input matrix is divided in horizontal stripes, each one is assigned to a thread (**jltb/jutb**).

Magic numbers:
2=half of the (stencil size-1)
5=stencil size.

Use loop bounds computed with thread ids instead of **for** loop directive

```
#pragma omp parallel
```

```
{
```

```
int thread_id = omp_get_thread_num();
int nthreads = omp_get_num_threads();

int jltb = 1 + (jmax-2) * ( thread_id ) / nthreads;
int jutb = 1 + (jmax-2) * ( thread_id + 1 ) / nthreads;

int ifltb = (jmax*imax*4) * ( thread_id ) / nthreads;
int ifutb = (jmax*imax*4) * ( thread_id + 1 ) / nthreads;

int jltb0 = jltb;
if (thread_id == 0) jltb0--;
int jutb0 = jutb;
if (thread_id == nthreads-1) jutb0++;

int kmin = MAX(jmax/2-5,jltb);
int kmax = MIN(jmax/2+5,jutb);
```

```
if (thread_id == 0) cpu_timer_start(&tstart_init);
```

```
for (int j = jltb0; j < jutb0; j++){
    for (int i = 0; i < imax; i++){
        xnew[j][i] = 0.0;
        x[j][i] = 5.0;
    }
}
```

```
for (int j = kmin; j < kmax; j++){
    for (int i = imax/2 - 5; i < imax/2 -1; i++){
        x[j][i] = 400.0;
    }
}
```

```
#pragma omp barrier
```

```
if (thread_id == 0) init_time += cpu_timer_stop(tstart_init);
```

Computation of the loop bounds.
The input matrix is divided in horizontal stripes, each one is assigned to a thread (**jltb/jutb**).

Magic numbers:

2=half of the (stencil size-1)

5=stencil size.

Use loop bounds computed with thread ids instead of **for** loop directive

Add barrier to avoid race condition.
This is not required with for loop thanks to the implied barrier

qui serve scriverlo perchè non lo fa da solo

```
for (int iter = 0; iter < 10000; iter++){
    if (thread_id == 0) cpu_timer_start(&tstart_flush);
    for (int l = ifltb; l < ifutb; l++){
        flush[l] = 1.0;
    }
    if (thread_id == 0){
        flush_time += cpu_timer_stop(tstart_flush);
        cpu_timer_start(&tstart_stencil);
    }
    for (int j = jltb; j < jutb; j++){
        for (int i = 1; i < imax-1; i++){
            xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j-1][i] + x[j+1][i] )/5.0;
        }
    }
    #pragma omp barrier
    if (thread_id == 0){
        stencil_time += cpu_timer_stop(tstart_stencil);

        SWAP_PTR(xnew, x, xtmp);
        if (iter%1000 == 0) printf("Iter %d\n",iter);
    }
    #pragma omp barrier
}
} // end omp parallel
```



```
for (int iter = 0; iter < 10000; iter++){
    if (thread_id == 0) cpu_timer_start(&tstart_flush);
    for (int l = ifltb; l < ifutb; l++){
        flush[l] = 1.0;
    }
    if (thread_id == 0){
        flush_time += cpu_timer_stop(tstart_flush);
        cpu_timer_start(&tstart_stencil);
    }
    for (int j = jltb; j < jutb; j++){
        for (int i = 1; i < imax-1; i++){
            xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j-1][i] + x[j+1][i] )/5.0;
        }
    }
    #pragma omp barrier
    if (thread_id == 0){
        stencil_time += cpu_timer_stop(tstart_stencil);

        SWAP_PTR(xnew, x, xtmp);
        if (iter%1000 == 0) printf("Iter %d\n",iter);
    }
    #pragma omp barrier
}
} // end omp parallel
```

Use loop bounds computed with thread ids instead of **for** loop directive

```

for (int iter = 0; iter < 10000; iter++){
    if (thread_id == 0) cpu_timer_start(&tstart_flush);
    for (int l = ifltb; l < ifutb; l++){
        flush[l] = 1.0;
    }
    if (thread_id == 0){
        flush_time += cpu_timer_stop(tstart_flush);
        cpu_timer_start(&tstart_stencil);
    }
    for (int j = jltb; j < jutb; j++){
        for (int i = 1; i < imax-1; i++){
            xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j
        }
    }
    #pragma omp barrier
    if (thread_id == 0){
        stencil_time += cpu_timer_stop(tstart_stencil);

        SWAP_PTR(xnew, x, xtmp);
        if (iter%1000 == 0) printf("Iter %d\n",iter);
    }
    #pragma omp barrier
}
} // end omp parallel

```

Use loop bounds computed with thread ids instead of **for** loop directive

Use loop bounds computed with thread IDs instead of **for** loop directive


```

for (int iter = 0; iter < 10000; iter++){
    if (thread_id == 0) cpu_timer_start(&tstart_flush);
    for (int l = ifltb; l < ifutb; l++){
        flush[l] = 1.0;
    }
    if (thread_id == 0){
        flush_time += cpu_timer_stop(tstart_flush);
        cpu_timer_start(&tstart_stencil);
    }
    for (int j = jltb; j < jutb; j++){
        for (int i = 1; i < imax-1; i++){
            xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j
        }
    }
    #pragma omp barrier
    if (thread_id == 0){
        stencil_time += cpu_timer_stop(tstart_stencil);

        SWAP_PTR(xnew, x, xtmp);
        if (iter%1000 == 0) printf("Iter %d\n",iter);
    }
    #pragma omp barrier
}
} // end omp parallel

```

Use loop bounds computed with thread ids instead of **for** loop directive

Use loop bounds computed with thread IDs instead of **for** loop directive

Add barrier to avoid race condition. This is not required with for loop thanks to the implied barrier

```
for (int iter = 0; iter < 10000; iter++){
    if (thread_id == 0) cpu_timer_start(&tstart_flush);
    for (int l = ifltb; l < ifutb; l++){
        flush[l] = 1.0;
    }
    if (thread_id == 0){
        flush_time += cpu_timer_stop(tstart_flush);
        cpu_timer_start(&tstart_stencil);
    }
    for (int j = jltb; j < jutb; j++){
        for (int i = 1; i < imax-1; i++){
            xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j-1][i] + x[j+1][i] );
        }
    }
    #pragma omp barrier
    if (thread_id == 0){
        stencil_time += cpu_timer_stop(tstart_stencil);

        SWAP_PTR(xnew, x, xtmp);
        if (iter%1000 == 0) printf("Iter %d\n",iter);
    }
    #pragma omp barrier
}
} // end omp parallel
```

Use loop bounds computed with thread ids instead of **for** loop directive

Use loop bounds computed with thread IDs instead of **for** loop directive

Add barrier to avoid race condition. This is not required with for loop thanks to the implied barrier

Uses thread ID instead of master/masked to reduce synchronization

For loop and SIMD

- OpenMP threaded loops can be combined with SIMD instructions to further improve the performance:
- #pragma omp [parallel] for simd
#pragma omp simd

```
#pragma omp for
for (int j = 1; j < jmax-1; j++){
    for (int i = 1; i < imax-1; i++){
        xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j-1][i] + x[j+1][i] )/5.0;
    }
}
```

```
#pragma omp for
for (int j = 1; j < jmax-1; j++){
    #ifdef OMP_SIMD
    #pragma omp simd
    #endif
    for (int i = 1; i < imax-1; i++){
        xnew[j][i] = ( x[j][i] + x[j][i-1] + x[j][i+1] + x[j-1][i] + x[j+1][i] )/5.0;
    }
}
```

A more complex example

- Let us consider a more complex example using a stencil that can be computed separately along its x and y axis
 - E.g. a cross shaped kernel
 - E.g. a Gaussian kernel is separable and we could compute it using two 1D kernels

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

Memory requirements

- Considering this case where we need to compute along two different faces we notice that we have different requirements to memory access.
- Considering a distribution of the data along horizontal stripes, the computations along x follow the data distribution
 - Each thread can use private and local data
- Instead, computations along y need data distributed among different threads,
 - Need shared data
- Let's use as much as possible private and local data

Data locality

- The first touch principle of most kernels says that memory will most likely be local to the thread (except at the edges between threads on page boundaries).
- We can improve the memory locality by making the array sections completely private to the thread where possible, such as the x-oriented data.
- Increasing the data locality is essential in minimizing the increasing speed gap between processors and memory.

Sequential implementation

```
void SplitStencil(double **a, int imax, int jmax)
{
    double** xface = malloc2D(jmax, imax);
    double** yface = malloc2D(jmax, imax);
    for (int j = 1; j < jmax-1; j++){
        for (int i = 0; i < imax-1; i++){
            xface[j][i] = (a[j][i+1]+a[j][i])/2.0;
        }
    }
    for (int j = 0; j < jmax-1; j++){
        for (int i = 1; i < imax-1; i++){
            yface[j][i] = (a[j+1][i]+a[j][i])/2.0;
        }
    }
    for (int j = 1; j < jmax-1; j++){
        for (int i = 1; i < imax-1; i++){
            a[j][i] = (a[j][i]+xface[j][i]+xface[j][i-1]+yface[j][i]+yface[j-1][i])/5.0;
        }
    }
    free(xface);
    free(yface);
}
```

- Let's consider a cross shaped stencil that computes values on the faces of the cells.
- X-face calc needs adjacent cells in the x direction
- Y-face calc needs adjacent cells in the y direction
- We add contributions from x and y faces for the final result

OpenMP and memory

- Each thread computes x-face data in a local variable
- Y-face data is computed in a shared scope variable, since threads operating on border elements need to access data from other threads
 - Race condition: it must have been already computed !
- There's no race condition in writing the final results, each thread operates on its cells
 - Perhaps there's risk of cache false sharing
- Remind the thread scoping of OpenMP:

Scoping rules

Specification		Shared	Private	Reduction
Parallel construct		Variables declared outside parallel construct or in a shared clause	Automatic variables within parallel construct or in a private or firstprivate clause	Reduction clause
Parallel region	Fortran routine	save attribute, initialized variables, common block, module variables	All local variables or declared threadprivate. Also dynamically allocated	
	C routine	File scope variables, extern, or static		
	Arguments	Inherited from calling environment		
Always		All loop indices will be private		

Parallelization

- Similar to the previous example initialize the input matrix in a `#pragma omp parallel` or `#pragma omp parallel for` section
 - Prefer the first one... compute the indices to allocate matrix parts next to the cores
- Execute the function computing the stencil within a `#pragma omp parallel` section
 - Could be the section used to allocate the input matrix

Function parallelization

```
void SplitStencil(double **a, int imax, int jmax)
{
    int thread_id = omp_get_thread_num();
    int nthreads = omp_get_num_threads();

    int jltb = 1 + (jmax-2) * ( thread_id      ) / nthreads;
    int jutb = 1 + (jmax-2) * ( thread_id + 1 ) / nthreads;

    int jfltb = jltb;
    int jfutb = jutb;
    if (thread_id == 0) jfltb--;

    double** xface = (double **)malloc2D(jutb-jltb, imax-1);
    static double** yface;
    if (thread_id == 0) yface = (double **)malloc2D(jmax+2, imax);
#pragma omp barrier
    for (int j = jltb; j < jutb; j++){
        for (int i = 0; i < imax-1; i++){
            xface[j-jltb][i] = (a[j][i+1]+a[j][i])/2.0;
        }
    }
    for (int j = jfltb; j < jfutb; j++){
        for (int i = 1; i < imax-1; i++){
            yface[j][i] = (a[j+1][i]+a[j][i])/2.0;
        }
    }
#pragma omp barrier
    for (int j = jltb; j < jutb; j++){
        for (int i = 1; i < imax-1; i++){
            a[j][i] = (a[j][i]+xface[j-jltb][i]+xface[j-jltb][i-1]+
                      yface[j][i]+yface[j-1][i])/5.0;
        }
    }
    free(xface);
#pragma omp barrier
    if (thread_id == 0) free(yface);
}
```

- Compute the indexes to assign the chunks of input data to the threads
- Initialize local memory to store x-face data
- Initialize shared memory to store y-face data
 - Static variable
 - Synchronize with a barrier or threads may start before the shared memory is ready !
- Compute x-face data
- Compute y-face data
 - Synchronize with a barrier or threads may start computing the final value w/o y-face data !
- Use of local memory may lead to super-linear speedup !

Function parallelization: details

```
void SplitStencil(double **a, int imax, int jmax)
{
    int thread_id = omp_get_thread_num();
    int nthreads = omp_get_num_threads();

    int jltb = 1 + (jmax-2) * ( thread_id      ) / nthreads;
    int jutb = 1 + (jmax-2) * ( thread_id + 1 ) / nthreads;

    int jfltb = jltb;
    int jfutb = jutb;
    if (thread_id == 0) jfltb--;

    double** xface = (double **)malloc2D(jutb-jltb, imax-1);
    static double** yface;
    if (thread_id == 0) yface = (double **)malloc2D(jmax+2, imax);
    #pragma omp barrier
```

- xface is allocated on the thread stack. It's local and private. It's small: each thread needs to store only the data required for it's computation.
- yface is static therefore shared. Only one thread needs to allocate it. It's allocation **MUST** be synchronized with the barrier. It's large to contain all values.

Function parallelization: details

```
#pragma omp barrier
for (int j = jltb; j < jutb; j++){
    for (int i = 0; i < imax-1; i++){
        xface[j-jltb][i] = (a[j][i+1]+a[j][i])/2.0;
    }
}

for (int j = jfltb; j < jfutb; j++){
    for (int i = 1; i < imax-1; i++){
        yface[j][i] = (a[j+1][i]+a[j][i])/2.0;
    }
}

#pragma omp barrier
```

- Computing xface does not require synchronization. It's a local and private variable
- yface must be ready before it's used in the following stage where the final value is computed. This requires synchronization, or we'll have a race condition in the following loop.

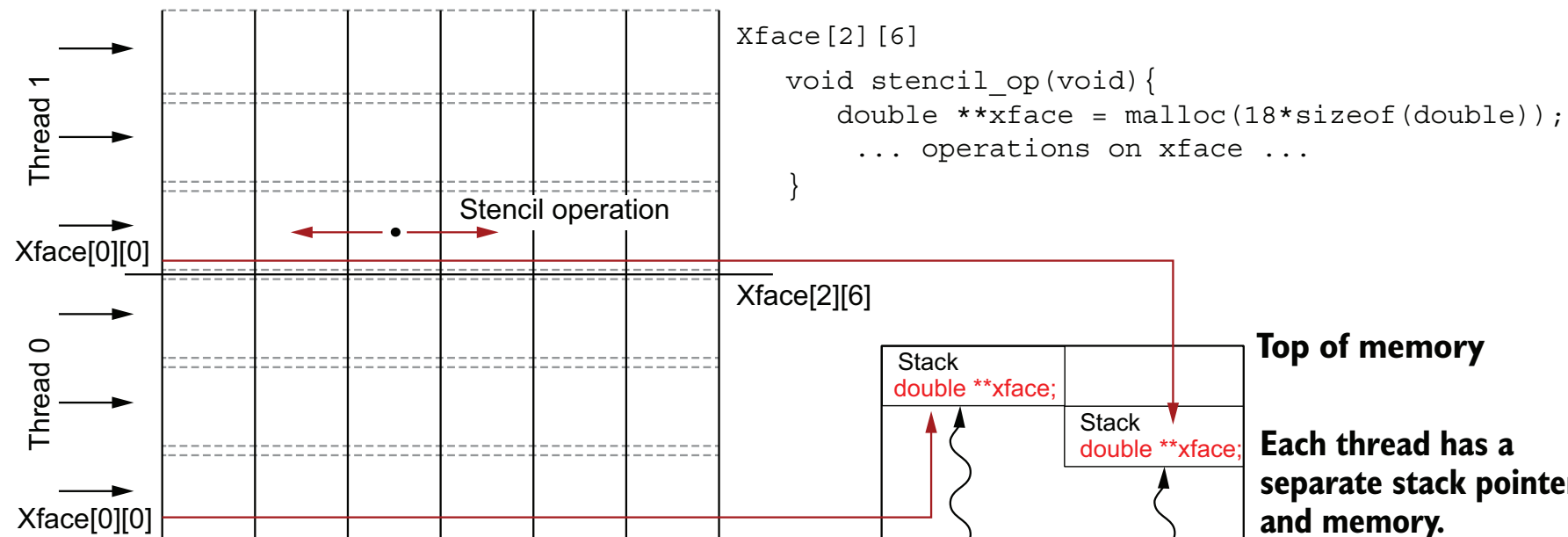
Function parallelization: details

- Since yface is surely fully computed we compute the final values.
- Access to xface is fast, it's a local memory, access to yface may not be local.
- Before freeing yface we must be sure that all threads have completed their computation, thus we need a final barrier.

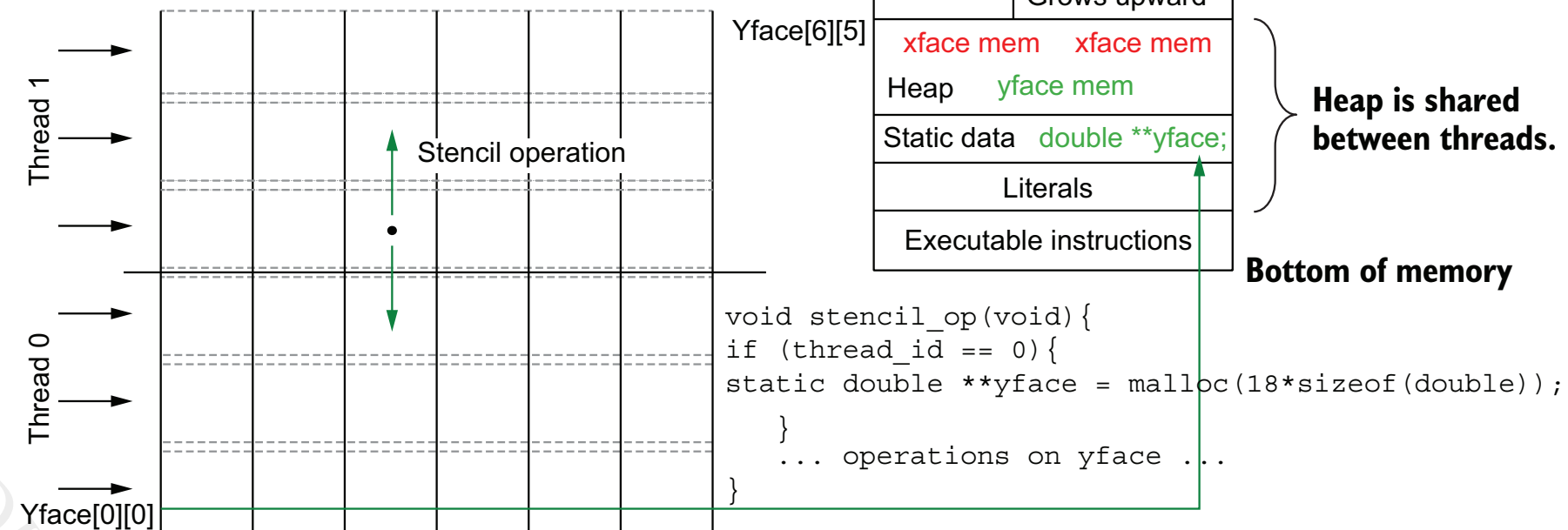
```
#pragma omp barrier
for (int j = jltb; j < jutb; j++){
    for (int i = 1; i < imax-1; i++){
        a[j][i] = (a[j][i]+xface[j-jltb][i]+xface[j-jltb][i-1]+
                    yface[j][i]+yface[j-1][i])/5.0;
    }
}
free(xface);
#pragma omp barrier
if (thread_id == 0) free(yface);
}
```

Memory view

Xface variable memory distribution



Yface variable memory distribution



Credits

- These slides report material from:
 - Ruud van der Pas (Oracle)

Books

- Parallel and High Performance Computing, R. Robey, Y. Zamora, Manning - Chapt. 7 and 14



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Parallel Computing

Prof. Marco Bertini



Shared memory: C/C++ Thread sanitizer

Thread Sanitizer (TSan)

- The Thread Sanitizer, or TSan, is an LLVM based tool for C/C++ languages that detects data races at runtime.
 - Data races occur when multiple threads access the same memory without synchronization and at least one access is a write.
 - Data races are dangerous because they can cause programs to behave unpredictably, or even result in memory corruption.
- TSan also detects other threading bugs, including uninitialized mutexes and thread leaks. It can also detect deadlocks.

Thread Sanitizer (TSan)

- The Thread Sanitizer, or TSan, is an LLVM based tool for C/C++ languages that detects data races at runtime.

Works also on Go and Swift

- Data races occur when multiple threads access the same memory without synchronization and at least one access is a write.
- Data races are dangerous because they can cause programs to behave unpredictably, or even result in memory corruption.
- TSan also detects other threading bugs, including uninitialized mutexes and thread leaks. It can also detect deadlocks.

Data race

Difficile da capire con un debugger

- Two threads access the same shared variable
 - at least one thread modifies the variable
 - the accesses are concurrent, i.e. unsynchronized
- Leads to non-deterministic behavior
- Hard to find with traditional debugging tools



Other sanitizers

- There are also other sanitizers (that should be used) for C/C++:
 - Address Sanitizer (ASAN, `-fsanitize=address`): detect out-of-bounds memory accesses.
 - Memory Sanitizer (MSAN, `-fsanitize=memory`): detect reads of uninitialized memory.
 - Undefined Behaviour Sanitizer (UBSAN, `-fsanitize=undefined`): detect reads of uninitialized memory.
 - Leak Sanitizer (LSAN, `-fsanitize=leak`): detect memory leaks (recent Clang versions enable this by default when ASAN is used).

Other sanitizers

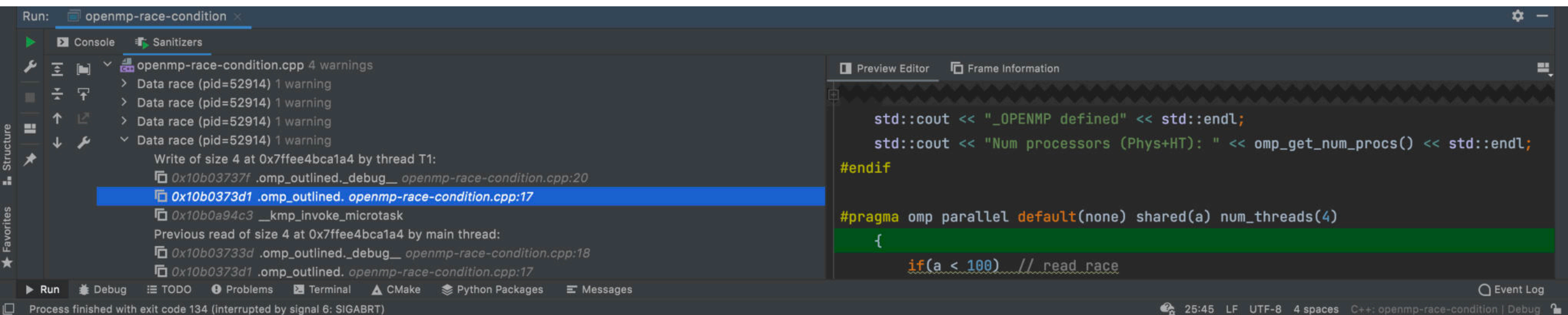
- There are also other sanitizers (that should be used) for C/C++.

All sanitizers (except MSAN) are available in GCC version ≥ 5

- Address Sanitizer (ASAN, `-fsanitize=address`): detect out-of-bounds memory accesses.
- Memory Sanitizer (MSAN, `-fsanitize=memory`): detect reads of uninitialized memory.
- Undefined Behaviour Sanitizer (UBSAN, `-fsanitize=undefined`): detect reads of uninitialized memory.
- Leak Sanitizer (LSAN, `-fsanitize=leak`): detect memory leaks (recent Clang versions enable this by default when ASAN is used).

IDE and sanitizers

- JetBrains CLion parses the output of the sanitizers and show them in the GUI



How TSan Works

- The Thread Sanitizer records the information about each memory access, and checks whether that access participates in a race. All memory accesses in the code is transformed by the compiler in the following way:
- Pseudocode for Thread Sanitizer memory access
- `// Before` Guarda se vengono modificate da più variabili stesse zone di memoria
- `*address = ...; // or: ... = *address;`
- `// After`
- `RecordAndCheckWrite(address);`
- `*address = ...; // or: ... = *address;`
- Each thread stores its own timestamp and the timestamps for other threads in order to establish points of synchronization. Timestamps are incremented each time memory is accessed. By checking for consistency of memory access across threads, data races can be detected independent of the actual timing of the access. Therefore, the Thread Sanitizer can detect races even if they didn't manifest during a particular run.

How TSan Works

- The Thread Sanitizer records the information about each memory access, and checks whether that access participates in a race. All memory accesses in the code is transformed by the compiler in the following way:
- Pseudocode for Thread Sanitizer memory access
- `// Before`
- `*address = ...; // or: ... = *address;`
- `// After`

Run disabling the ASLR (Address Space Layout Randomization) or TSAN will fail !

E.g.: `setarch $(name -m) -R ./my-executable`

disables (-R) the Linux ASLR

Occhio a disabilitare solo il programma e non l'intero sistema

- Each thread stores its own timestamp and the timestamps for other threads in order to establish points of synchronization. Timestamps are incremented each time memory is accessed. By checking for consistency of memory access across threads, data races can be detected independent of the actual timing of the access. Therefore, the Thread Sanitizer can detect races even if they didn't manifest during a particular run.

Performance Impact

- Running your code with Thread Sanitizer checks enabled can result in CPU slowdown of 2x to 20x, and an increase in memory usage by 5x to 10x.
- You can improve memory utilization and CPU overhead by compiling at the -O1 / -O2 optimization level.
- Example:

```
clang++ -fsanitize=thread -std=c++11  
-pthread -g -O1
```

Performance Impact

- Running your code with Thread Sanitizer checks enabled can result in CPU slowdown of 2x to 20x, and an increase in memory usage by 5x to 10x.
- You can improve memory utilization and CPU overhead by compiling at the -O1 / -O2 optimization level.
- Example:

```
clang++ -fsanitize=thread -std=c++11  
-pthread -g -O1
```

-g to get file names and line numbers in the warning messages.

-O1 for performance

Performance Impact

- Running your code with Thread Sanitizer checks enabled can result in CPU slowdown of 2x to 20x, and an increase in memory usage by 5x to 10x.
- You can improve memory utilization and CPU overhead by compiling at the -O1 / -O2 optimization level.

- Example:

-fsanitize=thread is both a compiler and linker parameter !

```
clang++ -fsanitize=thread -std=c++11  
-pthread -g -O1
```

-g to get file names and line numbers in the warning messages.

-O1 for performance

Using TSan

- Errors detected by sanitizers are caught at runtime.
- Execute the program compiled with the sanitizer and check them, eg.:

```
% ./a.out
```

```
WARNING: ThreadSanitizer: data race (pid=19219)
```

```
Write of size 4 at 0x7fcf47b21bc0 by thread T1:
```

```
#0 Thread1 tiny_race.c:4 (exe+0x00000000a360)
```

```
Previous write of size 4 at 0x7fcf47b21bc0 by main thread:
```

```
#0 main tiny_race.c:10 (exe+0x00000000a3b4)
```

```
Thread T1 (running) created at:
```

```
#0 pthread_create tsan_interceptors.cc:705 (exe+0x00000000c790)
```

```
#1 main tiny_race.c:9 (exe+0x00000000a3a4)
```


Archer

- Archer is a data race detector for OpenMP programs.
- Starting with LLVM/10, the Archer runtime is included in LLVM releases.
- Add `-fsanitize=thread` to the OpenMP switches (compiler and linker)



Intel Inspector

- Intel Inspector is a tool with a graphical user interface that is effective at detecting race conditions and deadlocks in OpenMP code.
- It is an Intel proprietary tool, but it is now freely available. It can be installed from the OneAPI suite from Intel



Links

- <https://clang.llvm.org/docs/ThreadSanitizer.html>
- <https://github.com/google/sanitizers/wiki/ThreadSanitizerCppManual>
- <https://www.intel.com/content/www/us/en/developer/tools/oneapi/inspector.html>

Books

- Parallel and High Performance Computing, R. Robey, Y. Zamora, Manning - Chapt. 17



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Parallel Computing

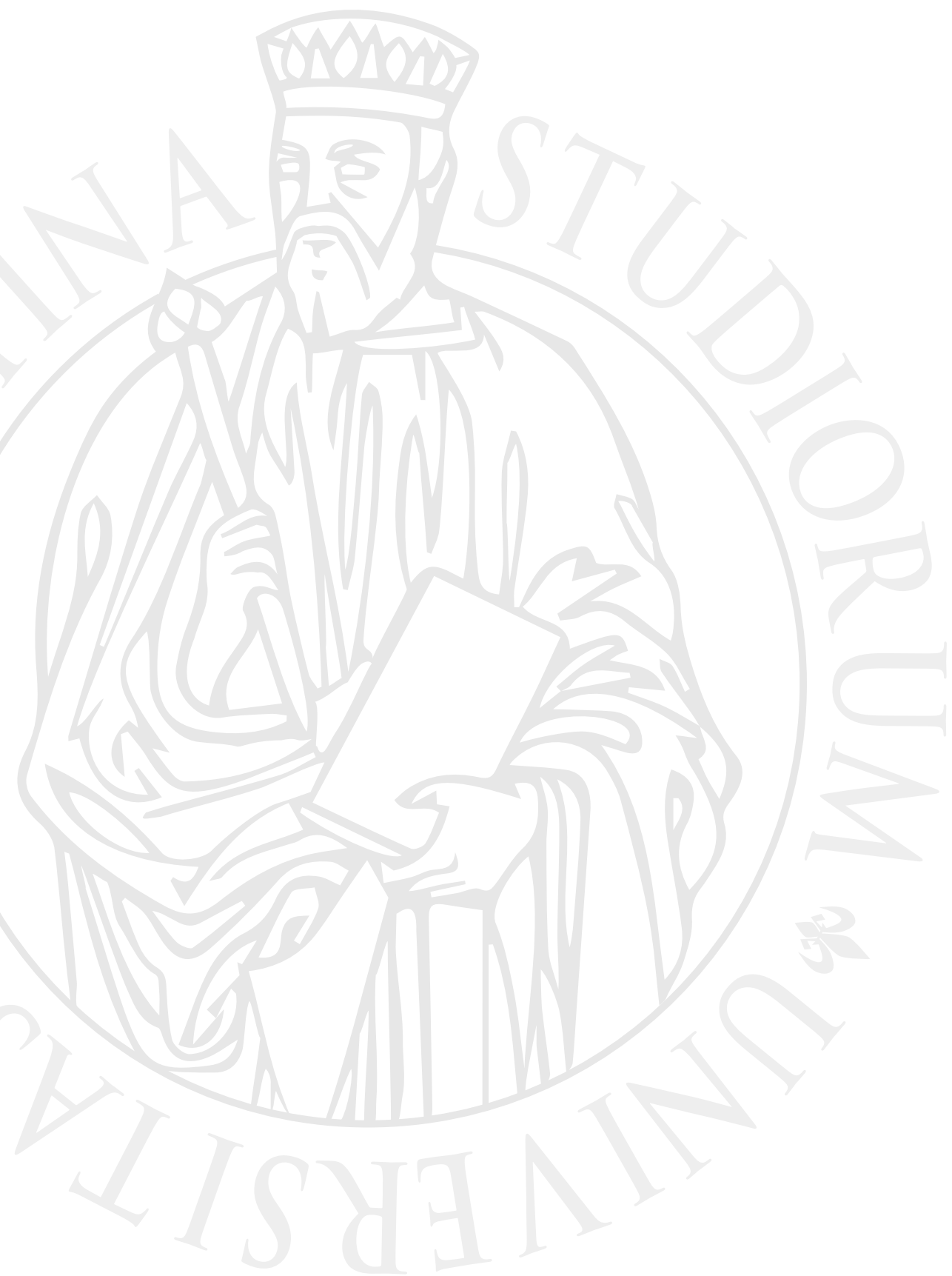
Prof. Marco Bertini



Shared memory: timing & profiling



UNIVERSITÀ
DEGLI STUDI
FIRENZE



Timing

Timing

- A simple approach to profile code is to add timers.
- Let us consider C code, using standard library.
- We need to evaluate how much time was needed to execute the sequential and parallel versions of our code.
- In `<time.h>` we find `clock()` and `time()`, and in `<sys/time.h>` we have `gettimeofday()`

clock()

- Returns the approximate **processor time** used by the process since the beginning of an implementation-defined era related to the program's execution. To convert result value to seconds, divide it by `CLOCKS_PER_SEC`.
- Only the difference between two values returned by different calls to `clock` is meaningful, as the beginning of the clock era does not have to coincide with the start of the program.

clock()

In a sequential program processor time $\cong$ program execution time.

- Returns the approximate **processor time** used by the process since the beginning of an implementation-defined era related to the program's execution. To convert result value to seconds, divide it by `CLOCKS_PER_SEC`.
- Only the difference between two values returned by different calls to `clock` is meaningful, as the beginning of the clock era does not have to coincide with the start of the program.

clock()

- In a parallel program `clock()` returns the total CPU time spent in all threads. If you run with four threads and each runs for $1/4$ of the time of a sequential program, `clock()` will still return the same value since $4 * (1/4) = 1$
- Often a thread has an overhead cost, so when using 4 threads the CPU time measured will be $> 1/4$ sequential time and `clock` will return a higher time difference for the parallel program than the sequential !

Do **NOT** use clock() to compare sequential and parallel execution time !

- In a parallel program `clock()` returns the total CPU time spent in all threads. If you run with four threads and each runs for $1/4$ of the time of a sequential program, `clock()` will still return the same value since $4 * (1/4) = 1$ Lui misura quanto le cpu sono occupate
- Often a thread has an overhead cost, so when using 4 threads the CPU time measured will be $> 1/4$ sequential time and `clock` will return a higher time difference for the parallel program than the sequential !

Elapsed real-time

- We need to evaluate how much time has passed, i.e. elapsed real-time, also known as: wall-clock time.
- Use `gettimeofday()` or its alternatives to get the wall clock.
- `c`lock time may advance faster or slower than the **wall clock**, depending on the execution resources given to the program by the operating system.
E.g., if the CPU is shared by other processes, `c`lock time may advance slower than wall clock. On the other hand, if the current process is multithreaded and more than one execution core is available, clock time may advance faster than wall clock.

gettimeofday()

- The `gettimeofday` system call gets the system's wall-clock time.
It obtains the current time, expressed as seconds and microseconds; the resolution of the system clock is unspecified.
- `time()` returns the wall-clock time from the OS, with precision in seconds.

According to POSIX standard `gettimeofday` is become obsolete and should be substituted with `clock_gettime`, that has nanosecond resolution.

gettimeofday()

- The `gettimeofday` system call gets the system's wall-clock time.
It obtains the current time, expressed as seconds and microseconds; the resolution of the system clock is unspecified.
- `time()` returns the wall-clock time from the OS, with precision in seconds.

According to POSIX standard `gettimeofday` is become obsolete and should be substituted with `clock_gettime`, that has nanosecond resolution.

Hint: check `clock_gettime` if available (e.g. it was introduced in macOS only in 10.12) otherwise fallback to `gettimeofday` or use a O.S. specific high res timer

Alternatives

- C++11 has introduced `std::chrono` with `::high_resolution_clock`
- OpenMP has a practical `omp_get_wtime()` which returns a double value of the number of seconds since an arbitrary point in the past. C'è il wall clock time





UNIVERSITÀ
DEGLI STUDI
FIRENZE

Profiling

Profiling

- Profiling is a technique that helps to identify, among other problems, which parts of our program are slower and perhaps could be improved by parallelization. Non tutte le parti del codice vanno parallelizzate
- There are several tools to identify how much time is spent in different parts of our programs. Typically they sample periodically the program, recording how many times an instruction has been executed and how much time it took to complete.
This info is logged to be analyzed at a later stage.

Profiling tools

- Depending on the language and framework used in the programs to be profiled we can choose the appropriate profiling tools
 - E.g. VisualVM or Yourkit for Java, Scalasca for OpenMP and MPI, VTune for OpenMP, C++, NVProf for CUDA, Google PerfTools and Callgrind for C/C++, Pthreads...
- There are different GUIs to analyze/visualize profiling results
 - E.g. IntelliJ IDEA for VisualVM and Yourkit, VTune itself, NVVP for CUDA, Cachegrind for Callgrind...

Profiling tools

- Depending on the language and framework used in the programs to be profiled we can choose the appropriate profiling tools
 - E.g. VisualVM or Yourkit for Java, Scalasca for OpenMP and MPI, VTune for OpenMP, C++, NVProf for CUDA, Google PerfTools and Callgrind for C/C++, Pthreads...
- For the sake of simplicity we are going to see only C/C++ profilers
- There are different GUIs to analyze/visualize profiling results
 - E.g. IntelliJ IDEA for VisualVM and Yourkit, VTune itself, NVVP for CUDA, Cachegrind for Callgrind...

Google perftools

- Google pprof is a tool available as part of the Google perftools package. It is used with Libprofiler, a sampling based profiler that is linked into your binary.
- It can be used in Linux and macOS.
- For Windows use other tools, e.g. Intel VTune.



CPU profiler

- After installing perftools:
 1. Link your executable with -lprofiler
 2. Run your executable with the CPUPROFILE environment var set:
`CPUPROFILE=/tmp/prof.out <path/to/binary> [binary args]`
 3. Run pprof to analyze the CPU usage logged in the /tmp/prof.out file:
`pprof --text <path/to/binary> /tmp/prof.out`

CPU profiler

- After installing perftools:

1. Link your executable with -lprofiler
2. Run your executable with the CPUPROFILE environment var set:
`CPUPROFILE=/tmp/prof.out <path/to/binary> [binary args]`

Set sampling rate (default: 100Hz) with CPUPROFILE_FREQUENCY
3. Run pprof to analyze the CPU usage logged in the /tmp/prof.out file:
`pprof --text <path/to/binary> /tmp/prof.out`

CPU profiler

- After installing perftools:

Hint: just add the library at execution time (no need to change build!), adding:

```
1 LD_PRELOAD=/path/to/libprofiler.so.0 (Linux)
  DYLD_INSERT_LIBRARIES=/path/to/libprofiler.dylib (macOS)
  to the line used to profile the program
```

2. Run your executable with the CPUPROFILE

environment var set:

```
CPUPROFILE=/tmp/prof.out <path/to/binary>
[binary args]
```

Set sampling rate (default: 100Hz) with CPUPROFILE_FREQUENCY

3. Run pprof to analyze the CPU usage logged in the /tmp/prof.out file:

```
pprof --text <path/to/binary> /tmp/
prof.out
```

Example

```
pprof --text ./a.out out.prof
```

```
... <snip> ...
```

Mi dicono quali funzoni ci mettono di più

```
Total: 311 samples
```

```
144 46.3% 46.3%
```

```
144 46.3% bar
```

```
95 30.5% 76.8%
```

```
95 30.5% foo
```

```
72 23.2% 100.0%
```

```
311 100.0% baz
```

```
0 0.0% 100.0%
```

```
311 100.0% __libc_start_main
```

```
0 0.0% 100.0%
```

```
311 100.0% _start
```

```
0 0.0% 100.0%
```

```
311 100.0% main
```

Example

In macOS the function names are not shown. The program has to be compiled disabling the address space layout randomization (ASLR) introduced in OS X 10.5

Add `-Wl,-no_pie` to the C/CXX flags.

Total: 311 samples

144 46.3% 46.3%

95 30.5% 76.8%

72 23.2% 100.0%

0 0.0% 100.0%

0 0.0% 100.0%

0 0.0% 100.0%

144 46.3% bar

95 30.5% foo

311 100.0% baz

311 100.0% __libc_start_main

311 100.0% _start

311 100.0% main

Analyzing Text Output

- Text mode has lines of output that look like this:
- 14 2.1% 17.2% 58 8.7% foo_bar
- The columns have the following meaning:
 - Number of profiling samples in this function
 - Percentage of profiling samples in this function
 - Percentage of profiling samples in the functions printed so far
 - Number of profiling samples in this function and its callees
 - Percentage of profiling samples in this function and its callees
 - Function name

Single procedure / lines

- It is possible to analyze a single function or get line by line analysis with:
 - `-list=function_name`
 - `-lines` (default report is at function level)



Example

```
pprof --list=thread_scan a.out a.out.prof
```

```
... <snip> ...
```

```
.      .  60: void* thread_scan(void* void_arg) {  
.      .  61:     // TODO(awreece) Copy locally so don't interfere with each other.  
.      .  62: thread_arg_t* args = (thread_arg_t*) void_arg;  
.      .  63: size_t i;  
.      .  64:  
303    323  65: for (i = 0; i < arg->size; i++) {  
6      10  66:     uint32_t val = arg->input[i];  
6      15  67:     if (val % 2 == 0) {  
9      300  68:         __sync_fetch_and_add(args->evens, 1);  
.      .  69:     }  
.      .  70: }  
.      .  71: }
```


Valgrind (Callgrind)

- Alternatively it is possible to use the Callgrind component of Valgrind
- It is (very much) slower than Google Perftools, but it is associated with nice GUIs to inspect the results (xCachegrind for QT/KDE)
- It is very accurate
- Like GPerftools does not require to recompile/link libraries, it instruments the executable

Running Callgrind

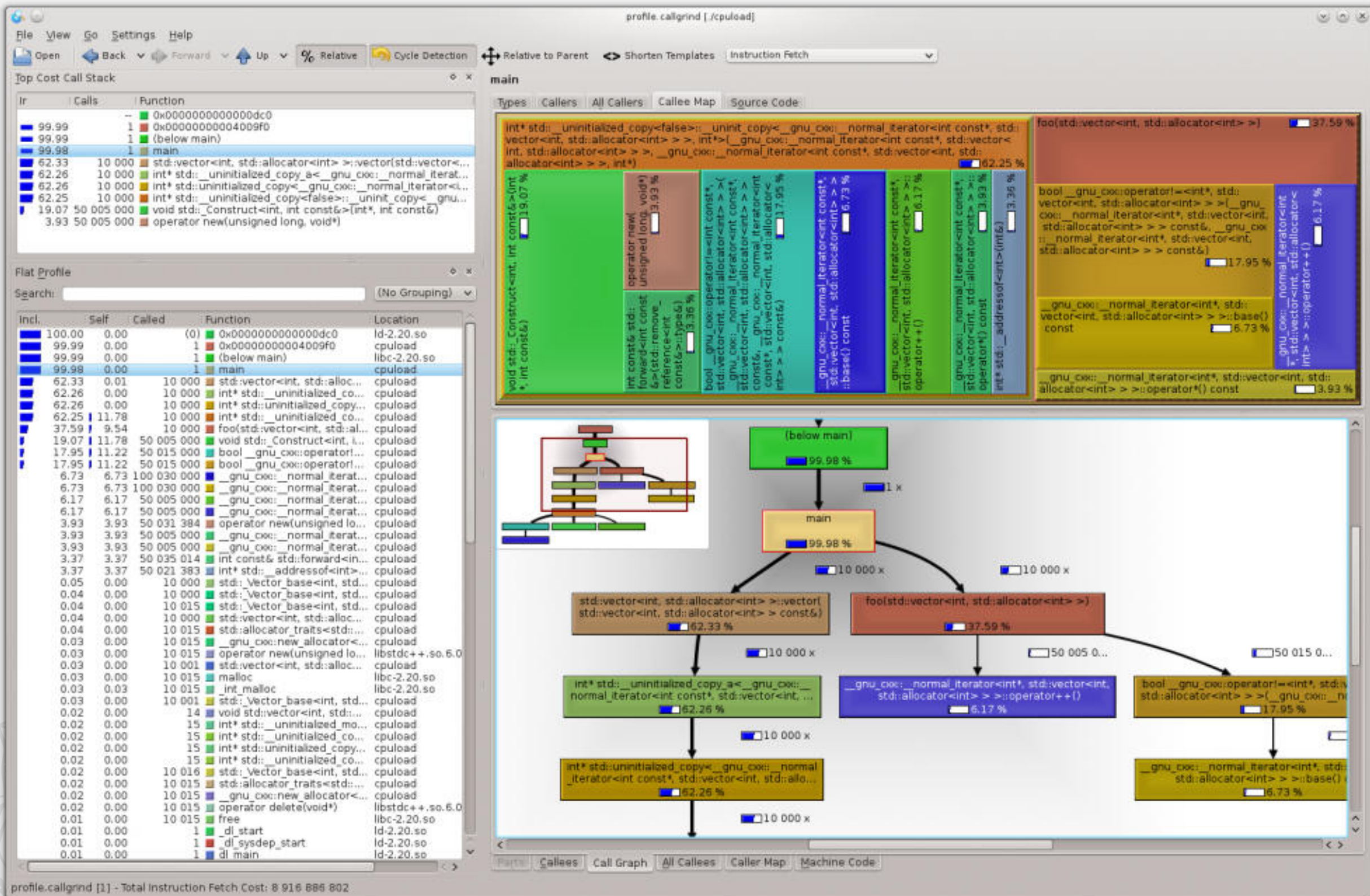
- `valgrind --tool=callgrind <path/to/binary>`
- It will create a `callgrind.out.<pid>` containing the sampling data, reporting how many “instructions read” events have been collected
- Analyze it using a xCachegrind GUI or
- Use `callgrind_annotate` command line tool to inspect it

Running Callgrind

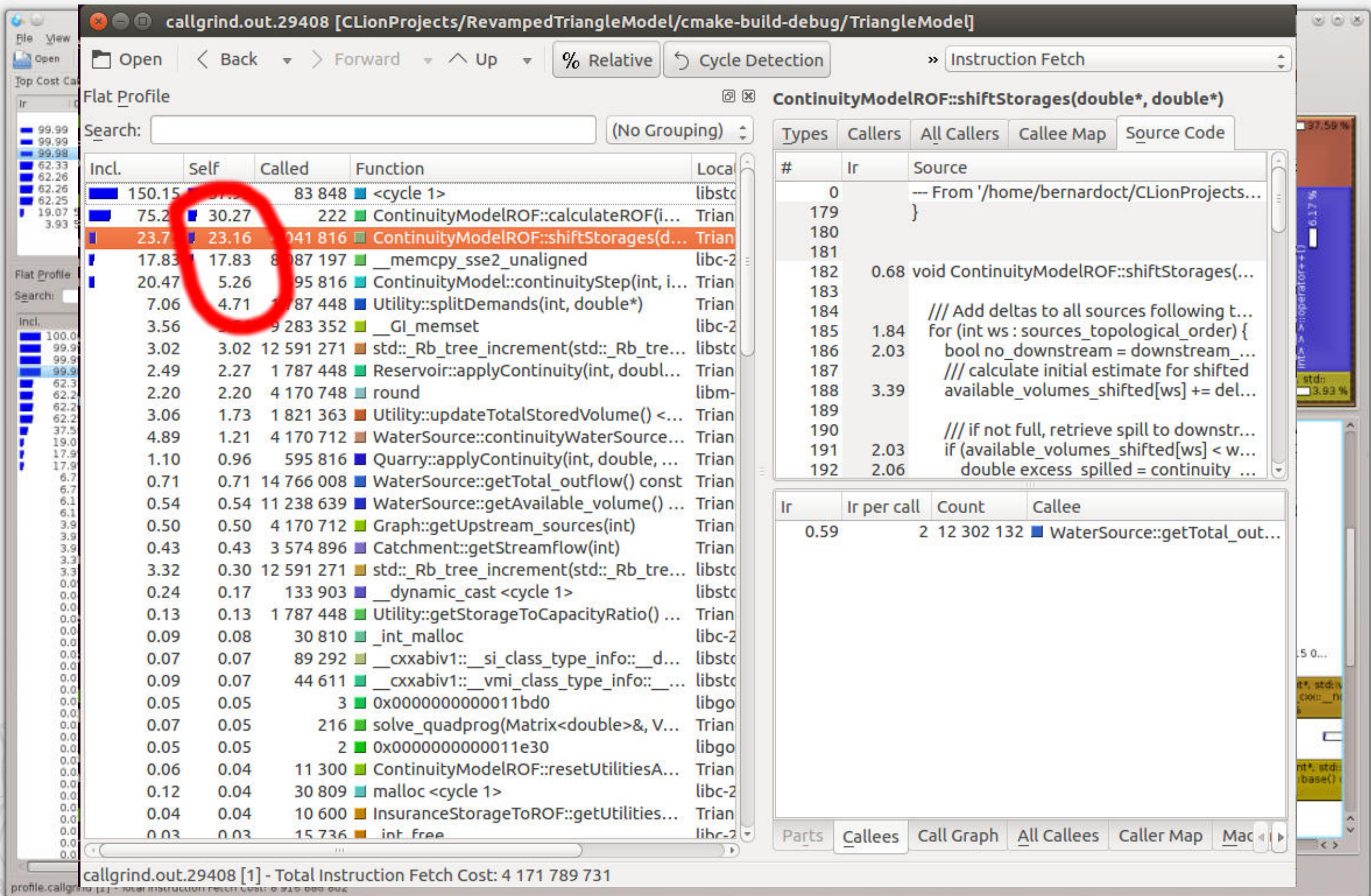
- `valgrind --tool=callgrind <path/to/binary>`
- It will create a `callgrind.out.<pid>` containing the sampling data, reporting how many “instructions read” events have been collected
- Analyze it using a xCachegrind GUI or
- Use `callgrind_annotate` command line tool to inspect it

Google Perftools provide Callgrind annotations format. Use `-callgrind` output specifier

Example: KCachegrind



Example: KCachegrind



Example: command line

- `callgrind_annotate --auto=yes
callgrind.out.<pid>`
- `-auto=yes` provides a line-based analysis,
otherwise only functions are analyzed
- The number reported next to each line is how many
times the line was executed
- A summary of most frequently called functions is
reported at the end

Profiling example

```
. void swap(int *a, int *b)
    3,000 {
    3,000     int tmp = *a;
    4,000     *a = *b;
    3,000     *b = tmp;
    2,000 }

.
. int find_min(int arr[], int start, int stop)
    3,000 {
    2,000     int min = start;
    2,005,000     for(int i = start+1; i <= stop; i++)
    4,995,000         if (arr[i] < arr[min])
    6,178             min = i;
    1,000     return min;
    2,000 }

. void selection_sort(int arr[], int n)
    3 {
    4,005     for (int i = 0; i < n; i++) {
    9,000         int min = find_min(arr, i, n-1);
    7,014,178 => sorts.c:find_min (1000x)
    10,000         swap(&arr[i], &arr[min]);
    15,000 => sorts.c:swap (1000x)
    .     }
    2 }
```

Cache profiling

- Callgrind can profile cache usage (as Cachegrind does).
- Add `--simulate-cache=yes`
- It is a simulation of L1/L2 caches, counting hits/misses
- It will report a summary printed at the end of the execution

Cache profiling example

```

==16409== Events      : Ir Dr Dw I1mr D1mr D1mw I2mr D2mr D2mw
==16409== Collected : 7163066 4062243 537262 591 610 182 16 103 94
==16409==
==16409== I    refs:      7,163,066
==16409== I1   misses:      591
==16409== L2i  misses:      16
==16409== I1   miss rate:    0.0%
==16409== L2i  miss rate:    0.0%
==16409==
==16409== D    refs:      4,599,505 (4,062,243 rd + 537,262 wr)
==16409== D1   misses:      792 (      610 rd +      182 wr)
==16409== L2d  misses:      197 (      103 rd +       94 wr)
==16409== D1   miss rate:    0.0% (      0.0% +      0.0% )
==16409== L2d  miss rate:    0.0% (      0.0% +      0.0% )
==16409==
==16409== L2   refs:      1,383 (      1,201 rd +       182 wr)
==16409== L2   misses:      213 (       119 rd +       94 wr)
==16409== L2   miss rate:    0.0% (      0.0% +      0.0% )

```

Cache profiling: output

- The cache simulator models a machine with a split L1 cache (separate instruction I1 and data D1), backed by a unified second-level cache (L2). This matches the general cache design of most modern CPUs.
- Ir: I cache reads (instructions executed)
- I1mr: I1 cache read misses (instruction wasn't in I1 cache but was in L2)
- I2mr: L2 cache instruction read misses (instruction wasn't in I1 or L2 cache, had to be fetched from memory)
- Dr: D cache reads (memory reads)
- D1mr: D1 cache read misses (data location not in D1 cache, but in L2)
- D2mr: L2 cache data read misses (location not in D1 or L2)
- Dw: D cache writes (memory writes)
- D1mw: D1 cache write misses (location not in D1 cache, but in L2)
- D2mw: L2 cache data write misses (location not in D1 or L2)

It is possible to get cache profiling at instruction line level

Tips and tricks

- When debugging we are used to compile in debug mode with no optimizations, but when profiling it is OK to test the optimized program
- These profilers record the count of instructions, not the actual time spent in a function. If you have a program where the bottleneck is file I/O, the costs associated with reading and writing files won't show up in the profile, as those are not CPU-intensive tasks.
- Evaluate using different input sizes.
- You can even profile at assembly level (with Callgrind)

Tips and tricks

- L2 misses are much more expensive than L1 misses, so pay attention to passages with high D2mr or D2mw counts.

You can use `callgrind_annotate` show/sort options to focus on key events:

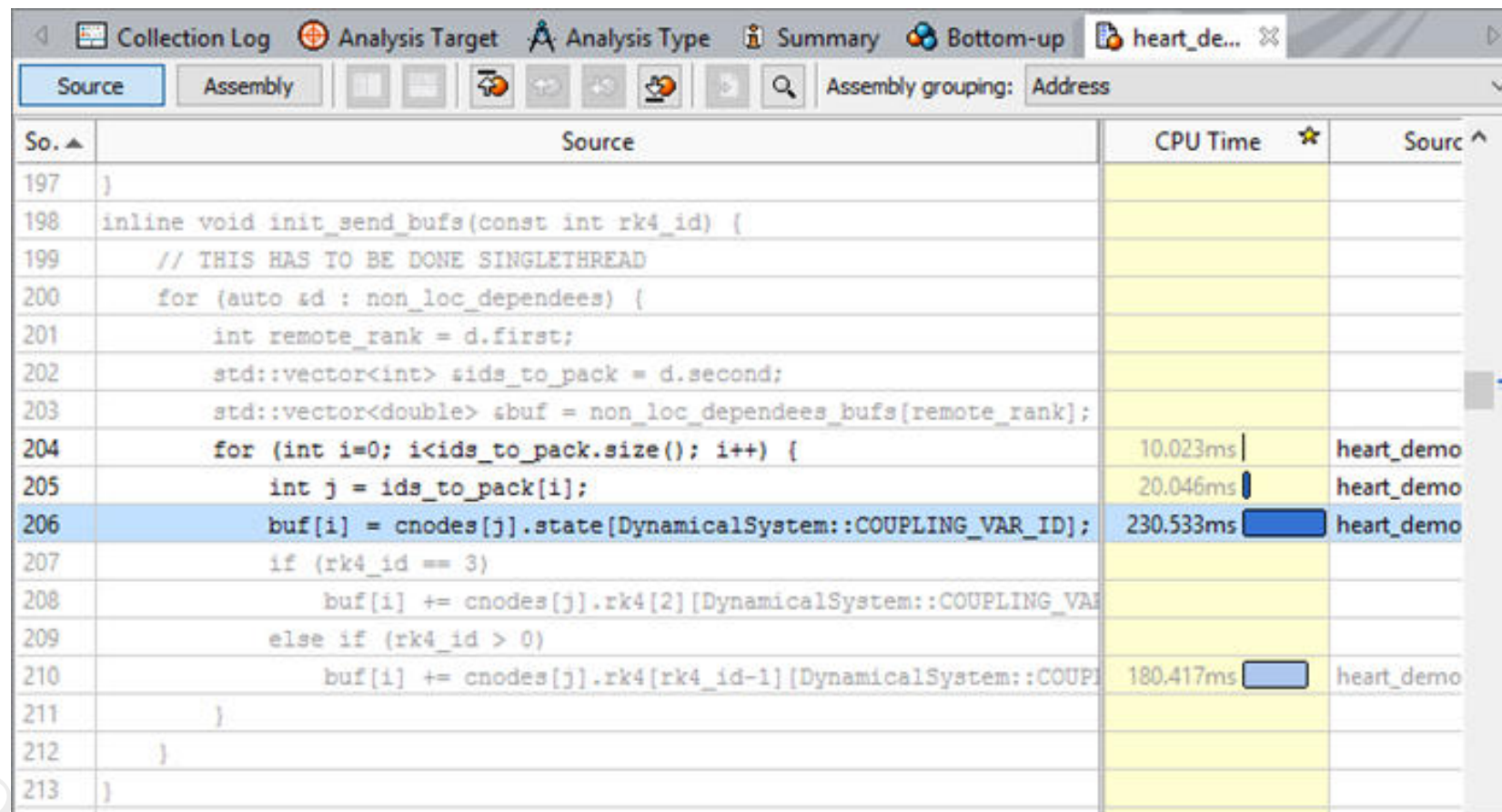
- `callgrind_annotate --show=D2mr`
`--sort=D2mr`

will highlight D2mr counts.

- These tools (PerfTools and Valgrind) have many parameters: check their manuals !

VTune

- Intel VTune is a general-purpose optimization tool that helps to identify bottlenecks and potential improvements.
- It is a proprietary tool but it is freely available for Windows and Linux. Part of oneAPI Toolkit.



The screenshot shows the Intel VTune Performance Analyzer interface. The 'Collection Log' tab is active, displaying a table of performance data for a collection named 'heart_demo'. The table has columns for 'So.' (Source), 'Source', 'CPU Time', and 'Source'. The 'CPU Time' column is highlighted in yellow, and the 'Source' column is highlighted in blue. The table shows several entries, with the most significant ones being 'buf[i] = cnodes[j].state[DynamicalSystem::COUPLING_VAR_ID];' (230.533ms) and 'buf[i] += cnodes[j].rk4[rk4_id-1][DynamicalSystem::COUPLING_VAR_ID];' (180.417ms).

So.	Source	CPU Time	Source
197	}		
198	inline void init_send_bufs(const int rk4_id) {		
199	// THIS HAS TO BE DONE SINGLETHREAD		
200	for (auto id : non_loc_dependees) {		
201	int remote_rank = d.first;		
202	std::vector<int> ids_to_pack = d.second;		
203	std::vector<double> sbuf = non_loc_dependees_bufs[remote_rank];		
204	for (int i=0; i<ids_to_pack.size(); i++) {	10.023ms	heart_demo
205	int j = ids_to_pack[i];	20.046ms	heart_demo
206	buf[i] = cnodes[j].state[DynamicalSystem::COUPLING_VAR_ID];	230.533ms	heart_demo
207	if (rk4_id == 3)		
208	buf[i] += cnodes[j].rk4[2][DynamicalSystem::COUPLING_VAR_ID];		
209	else if (rk4_id > 0)		
210	buf[i] += cnodes[j].rk4[rk4_id-1][DynamicalSystem::COUPLING_VAR_ID];	180.417ms	heart_demo
211	}		
212	}		
213	}		



UNIVERSITÀ
DEGLI STUDI
FIRENZE



Bottlenecks

Bottlenecks

- When dealing with threads and parallel processing remember these considerations:
 - I/O is a bottleneck.
 - Memory is usually a bottleneck.
 - Managing threads requires an overhead.



I/O Bottleneck

- Accessing I/O, whether it be a hard drive, data from a video camera, or sending through Bluetooth, will cause a thread to wait.
- Unless your program has something to do while waiting for I/O, parallel threads won't help here.

Memory Bottleneck / contention

- There may be contention when one core wants to access memory at the same time another core accesses memory. One thread (core) will have to wait for the other to finish (or interweave the requests). This contention may cause you to lose efficiency.



Overhead of Managing threads

- There is an overhead associated with the creation, maintenance and destruction of threads.
- A thread should perform enough execution to make the thread creation and maintenance worthwhile.
- Use a thread pool to reduce the cost of thread creation: instantiate them at the beginning and then re-use them during the execution of the parallel program.



Links

- Differences between C/C++ clocks: <https://stackoverflow.com/questions/12392278/measure-time-in-linux-time-vs-clock-vs-getrusage-vs-clock-gettime-vs-gettimeofday/12480485#12480485>
- Intel VTune: <https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler.html>

Books

- Parallel and High Performance Computing, R. Robey, Y. Zamora, Manning - Chapt. 3